

Manual técnico

Proyecto académico · Versión completa local · 15/09/2026

1. Arquitectura y estructura

```
Angular + Bootstrap (navegador)
  | HTTP JSON /api + cookie HttpOnly
Express: Origin/CORS -> autenticación -> validación -> rutas
  | consultas parametrizadas mysql2
MySQL: datos persistentes y sesiones
```

Angular maneja formularios y navegación. Express valida, autoriza y consulta. MySQL conserva todos los datos. No hay respuestas simuladas ni localStorage para información del dominio. En desarrollo un proxy une Angular con /api; en modo compilado Express sirve también los archivos del cliente.

Archivo / carpeta	Responsabilidad
frontend/src/app/core.ts	Tipos, servicio HTTP, sesión, guard e interceptor
frontend/src/app/routes.ts	Rutas públicas/protegidas
auth-page.ts	Registro, login y recuperación
feed.ts / post-card.ts	Listado, filtros, búsqueda y tarjetas
new-post.ts / post-detail.ts	Crear publicación y comentar
profile.ts	Perfil propio/ajeno y aprobados
backend/src/app.js	Configuración y montaje de rutas
backend/src/server.js	Escucha HTTP y cliente compilado
config.js / db.js	Variables de entorno y pool MySQL
auth.js	Cookie, token y autenticación
routes-*.js	Operaciones REST por área
validation.js	Esquemas Zod y validación de IDs
backend/scripts	Creación y semilla
backend/test	Integración MySQL y reinicio
database	SQL, catálogo con procedencia y usuario de ejemplo

2. Modelo relacional

Tabla	Clave / relaciones	Restricciones
users	PK id	Registro y correo UNIQUE; hash bcrypt
courses	PK id; código UNIQUE	Créditos sin signo; indicador de publicación
teachers	PK id	Nombre+rol UNIQUE; catedratico o auxiliar
assignments	FKs curso y docente	Curso/docente/semestre/sección UNIQUE
posts	FKs autor, curso, docente	CHECK exactamente un destino
comments	FKs publicación y autor	Mensaje y fecha
approved_courses	PK usuario+curso	Impide duplicados
sessions	PK token_hash; FK usuario	Caducidad de ocho horas

Un usuario tiene muchas publicaciones, comentarios, sesiones y aprobados. Una publicación tiene muchos comentarios. Usuarios y cursos aprobados forman una relación muchos-a-muchos. Cursos y docentes se relacionan mediante asignaciones por semestre y sección.

Las claves foráneas impiden referencias inexistentes. Índices adicionales permiten ordenar publicaciones, buscar comentarios y limpiar sesiones caducadas. Los créditos se consultan con JOIN, sin duplicarlos en el historial.

El esquema es versión inicial 1. db:setup no migra estructuras existentes, solo crea tablas ausentes; futuros cambios requieren migraciones explícitas.

3. Autenticación y permisos

Registro guarda bcrypt con costo 12 y sal individual. Login genera 32 bytes aleatorios; el navegador recibe una cookie HttpOnly/SameSite=Lax y MySQL guarda SHA-256 del token. Cada petición interna consulta sesión vigente y asigna req.user.

Los autores de publicaciones/comentarios provienen de req.user.id. Los cambios de perfil e historial comparan ese ID con el de la URL y rechazan recursos ajenos con 403. Los campos públicos excluyen el hash. Los objetos estrictos rechazan un intento de modificar el registro académico.

Logout elimina la sesión de MySQL. Recuperación actualiza contraseña e invalida sesiones dentro de una transacción. Los guards del cliente facilitan la navegación; la autorización efectiva siempre está en Express.

CORS permite el origen configurado; un middleware comprueba Origin en escrituras. Helmet agrega cabeceras, JSON se limita a 20 KB y las rutas de autenticación a 100 solicitudes por IP cada 15 minutos. NODE_ENV=production activa Secure. La app escucha en loopback.

La recuperación académica por registro y correo no prueba propiedad del correo. Para uso público requiere enlace de un solo uso u otra verificación, HTTPS y medidas operativas adicionales.

4. API REST

Base /api; JSON en entradas y salidas. Éxito 200 o 201 para creación. Error {message}; errores de validación añaden errors: [{field,message}]. IDs enteros positivos; registro académico como texto.

Rutas públicas

Método / ruta	Entrada	Respuesta	Errores
GET /health	Ninguna	status:ok tras SELECT 1	500 MySQL
POST /auth/register	academic_record, first_name, last_name, email, password	201 id,message	400 campos; 409 duplicados
POST /auth/login	academic_record,password	Usuario público y cookie	400; 401 credenciales
POST /auth/recover	academic_record,email,password nueva	message	400 identidad o contraseña
POST /auth/logout	{}	message y cookie expirada	500 persistencia

Rutas con sesión

Todas devuelven 401 sin sesión válida.

Método / ruta	Entrada	Respuesta	Otros errores
GET /auth/me	Ninguna	Usuario público	401
GET /catalogs	Ninguna	semester,courses[],teachers[],as signments[]	500
GET /posts	Query type=course/teacher, course_id, teacher_id, course_name, teacher_name (opcionales)	Publicaciones[] fecha/id DESC	400 filtros
POST /posts	message y exactamente course_id o teacher_id	201 publicación completa	400 destino/mensaje
GET /posts/:id	ID URL	Publicación	400 ID; 404
GET /posts/:id/comments	ID URL	Comentarios[] fecha/id ASC	400; 404
POST /posts/:id/comments	message	201 id	400; 404 publicación
GET /users/by-record/:record	Registro URL	Usuario público	400; 404
GET /users/:id	ID URL	Usuario,courses[],total_credits	400; 404
PATCH /users/:id	first_name,last_name,email	Usuario actualizado	403 ajeno; 400 campos extra; 409 correo
POST /users/:id/approved-courses	course_id	201 message	403 ajeno; 400 curso; 409 duplicado

Usuario público: id, academic_record, first_name, last_name, email. Publicación: id, author_id, course_id, teacher_id, message, created_at, academic_record, author_name, course_name, teacher_name, teacher_role, comment_count. Comentario: id, post_id, author_id, message, created_at, academic_record, author_name.

Ejemplo

```
{ "course_id": 16, "message": "¿Cómo recomiendan organizar las prácticas?" }
```

El ID es ilustrativo; obtenerlo de /catalogs. No enviar autor ni fecha. Para docente usar teacher_id en lugar de course_id.

5. Validación y SQL

- Nombres/apellidos: 1-80 caracteres tras quitar espacios extremos.
- Correo válido hasta 254 caracteres, normalizado a minúsculas.
- Registro: 5-20 dígitos, conserva ceros.
- Contraseña: al menos 8 caracteres, hasta 72 bytes para evitar truncamiento bcrypt.
- Publicación: 1-3000 caracteres útiles; comentario: 1-2000.
- Cursos publicados deben estar habilitados; docentes deben tener asignación del semestre.
- Perfil solo acepta tres campos editables; el registro es inmutable.
- Aprobados requieren curso existente y propietario autenticado.

Todos los valores de consultas usan ?. Las columnas dinámicas de filtros provienen de una lista cerrada. LOCATE permite búsquedas parciales sin interpretar comodines. El nombre de base del script de creación se valida antes de insertarlo en SQL.

6. Fechas, créditos y catálogos

MySQL fija fechas por DEFAULT CURRENT_TIMESTAMP. Cada conexión usa time_zone='+00:00' y mysql2 interpreta UTC. JSON transporta ISO 8601; Angular presenta la zona del navegador. La prueba compara la fecha de creación contra el reloj real para evitar doble conversión.

Los créditos se suman en el servidor a partir del JOIN con el catálogo. Se utiliza CLAR del plan 2025, sin mezclar el plan anterior. database/catalog.json conserva fecha y URLs de procedencia.

tools/import-catalog.py --download permite repetir extracción; requiere Python y BeautifulSoup4. Revisar cambios antes de cargar. La semilla conserva cursos existentes para no cambiar créditos retrospectivamente; las actualizaciones de plan requieren una migración revisada.

7. Ejecución y mantenimiento

Seguir el README para configuración, instalación y respaldo. Las semillas no sobrescriben contraseñas ni borran publicaciones. Para restaurar, usar una base nueva y conservar la anterior.

npm test usa MySQL real en opina_test y usuarios únicos. Incluye un proceso en 3101 que crea datos, se detiene y reinicia para verificar sesión, publicación, comentario y créditos. Nunca usar opina_test como base personal.

npm run build comprueba tipos y plantillas estrictos y limita tamaño del bundle. docs/verificacion.md registra los recorridos de navegador y las limitaciones reales.